

FUNCTIONS PRACTICE, RETURN, KEYWORD ARGUMENTS

Choose a better name for each of the four functions below. Each function is run with a single list as its input, e.g. `M1([3, 9, 0, 14])`

```
def M1(lst):  
    smallest = lst[0]  
    for elem in lst:  
        if elem < smallest:  
            smallest = elem  
    print(smallest)
```

```
def M2(lst):  
    running_sum = 0  
    for elem in lst:  
        running_sum += 1  
    print(running_sum)
```

```
def M3(lst):  
    saved = lst[0]  
    for elem in lst:  
        if elem > saved:  
            saved = elem  
    print(saved)
```

```
def M4(lst):  
    running_sum = 0  
    for elem in lst:  
        running_sum += elem  
    print(running_sum)
```

A: `max` B: `min` C: `sum` D: `len`

RECAP: CALLING FUNCTIONS WITH INPUTS

Here is a function that takes a message and a number and prints that message that number of times.

```
def print_n_times(msg, n):  
    counter = 0  
    while counter < n:  
        print(msg)  
        counter = counter + 1
```

What happens when we call the function: `print_n_times("Hi!", 3)`?

RECAP: CALLING FUNCTIONS WITH INPUTS

- The function's **parameters** are `msg` and `n`.
 - These are names for variables that can be used in the body of the function
- The function call provides two **arguments**: `"Hi!"` and `3`.
 - These are the values that the parameter variables will take at the call start.

```
# calling print_n_times("Hi!", 3)
def print_n_times(msg, n):
    # msg = "Hi!"
    # n = 3
    counter = 0
    while counter < n: # while counter < 3:
        print(msg)    # print("Hi!")
        counter = counter + 1
```

ACTIVITY: COUNTING NUMBERS

```
def add_three_numbers(a, b, c):  
    first_two = a + b  
    last = c + first_two  
    print(last)
```

- M5: calling the function as `add_three_numbers(3, 4, 7, 9)` leads the program to immediately crash
- M6: calling the function as `add_three_numbers("three", "four", "five")` leads the program to immediately crash

A: True, B: False

ACTIVITY: WORKING TOWARDS WRITING A FUNCTION

Assuming you have a list `lst` containing a bunch of numbers, write a couple of loops that print out all of the **negative** numbers and then all of the **non-negative numbers**. (C12, but leave just a little space at the top)

e.g.

```
lst = [9, -19, 31, -13, 1, 2]  
# TODO: Your loop(s) here
```



```
-19 -13 9 31 1 2
```

You're not writing a whole function yet! Just write some lines & loops like you've been doing before.

ACTIVITY: WORKING TOWARDS WRITING A FUNCTION

Write the signature for a function that prints out all of the **negative** numbers and then all of the **non-negative numbers**. (L15)

Remember: a signature consists of a `def`, a function name, and a list of parameters the function should be called with.

ACTIVITY: WORKING TOWARDS WRITING A FUNCTION

Add a signature to the code you wrote for (C14) in order to turn it into a function that can be called.

Then, in (C16), write an example of a function call that would print out the following output:

```
-30 -14 3 19 8
```


NEW: RETURN

Function calls are themselves *expressions*, meaning that they always have a value.

- The value of a function call is determined by the value that function **returns**

`return` is keyword that serves two purposes:

- stops function execution in its tracks
- provides a value for the expression of the function call

```
def multiply_two_numbers(a, b):  
    print(f"Multiplying {a} x {b}!")  
    product = a * b  
    return product
```

If we write the call `multiply_two_numbers(3, 7)` then...

```
# a = 3  
# b = 7  
print(f"Multiplying {a} x {b}!")  
product = a * b  
return product
```

*# product = 3 * 7*
return 21

...the following therefore evaluates to `True`

```
multiply_two_numbers(3, 7) == 21
```

PRINTING VS. RETURNING

An output that's *printed* is not the same as an output that's *returned*.

- Any call to `print()` will make text appear on the screen, but it doesn't produce a value
- If a function is supposed to calculate and create some value (e.g. the product of two numbers), it must *return* that value in the function body.

FUNCTIONS THAT HAVE NO RETURN

```
def our_min(lst):  
    smallest = lst[0]  
    for elem in lst:  
        if elem < smallest:  
            smallest = elem  
    print(smallest)
```

```
def our_len(lst):  
    running_sum = 0  
    for elem in lst:  
        running_sum += 1  
    print(running_sum)
```

```
some_numbers = [1000, 3, 8]
```

```
# 🖨️ ➡️ 3  
result = our_min(some_numbers)  
# 🖨️ ➡️ None  
print(result)
```

```
# 🖨️ ➡️ 3  
result = our_len(some_numbers)  
# 🖨️ ➡️ None  
print(result)
```

These functions both **compute** some value and then **print** it but do not **return** it.

ADDING RETURN

```
def our_min(lst):  
    smallest = lst[0]  
    for elem in lst:  
        if elem < smallest:  
            smallest = elem  
    return smallest
```

```
def our_len(lst):  
    running_sum = 0  
    for elem in lst:  
        running_sum += 1  
    return running_sum
```

```
some_numbers = [1000, 3, 8]
```

```
#   Nothing!  
result = our_min(some_numbers)  
#   3  
print(result)
```

```
#   Nothing!  
result = our_len(some_numbers)  
#   3  
print(result)
```

These functions now **compute** some value and then **return** it but do not **print** it.

THE POINT OF NO RETURN?

`return` works as a stopping/exit point for your program. If you execute a line with `return` you will leave that function call execution.

```
def print_all_above(lst, k):  
    for elem in lst:  
        if elem > k:  
            print(elem)  
  
print_all_above([5, 10, 15], 8)
```



10 15

THE POINT OF NO RETURN?

`return` works as a stopping/exit point for your program. If you execute a line with `return` you will leave that function call execution.

```
def print_first_above(lst, k):  
    for elem in lst:  
        if elem > k:  
            print(elem)  
            return  
  
print_first_above([5, 10, 15], 8)
```



THE POINT OF NO RETURN?

`return` works as a stopping/exit point for your program. If you execute a line with `return` you will leave that function call execution.

```
def return_first_above(lst, k):  
    for elem in lst:  
        if elem > k:  
            return elem  
  
return_all_above([5, 10, 15], 8)
```

...but it does return `10`

ACTIVITY

```
def foo(l):  
    for i, n in enumerate(l):  
        if n == i:  
            return n  
        if n == len(l):  
            print("🕒")  
    print("▶")
```

- What is the value of `x` if we run `x = foo([3, 1, 4])`? (S7)
- What values are printed if we run `x = foo([3, 1, 4])`? (S8)
- What is the value of `x` if we run `x = foo([10, 11, 12])`? (S9)
- What values are printed if we run `x = foo([3, 1, 4])`? (S10)

TYPE ANNOTATIONS

```
def return_first_above(lst: list[int], k : int) -> int:  
    for elem in lst:  
        if elem > k:  
            return elem
```

FIX THIS FUNCTION

C14 Rewrite the signature with type annotations; add a descriptive docstring comment:

```
def repeat_substring(text, substring, times):  
    """  
    Docstring description  
    input:  
    output:  
    """  
    result = ""  
    if substring in text:  
        for blah in range(times):  
            result += substring  
    return result
```

